Local vs. Cloud: A Comparative Study of On-Device and API-Based Large Language Model Agents for Desktop Automation

Prateek Sharma
Department of CSE
Manav Rachna International Institute
of Research and Studies
Faridabad, India
tkmhr17@gmail.com

Kushagra Agrawal
Department of CSE
Manav Rachna International Institute
of Research and Studies
Faridabad, India
agrawal_kushagra@rediffmail.com

***Abstract*—The choice between locally hosted and cloud-based Large Language Models (LLMs) is a central design decision for AI desktop agents, trading off privacy, latency, and offline availability against raw capability and ease of deployment. This paper presents a controlled comparative study of two functioning desktop automation agents that sit on opposite sides of this divide: ARIA, which performs all command interpretation locally using llama3 (8B) served through Ollama, and Mark-XXXIX, an open-source assistant that performs command interpretation through Google's Gemini API. Both systems expose comparable tool sets for file management, application control, and web interaction, allowing a like-for-like comparison along a single architectural axis: where the reasoning happens. We describe the implementation of each system as built, evaluate them on an identical set of 280 natural-language prompts spanning six task categories, and report task success rate, command interpretation accuracy, and response latency for each. We find that the local pipeline offers a meaningful privacy and offline-availability advantage and adequate accuracy for narrow, file-system-centric operations, while the cloud pipeline shows higher success on tasks requiring broad world knowledge or multi-step web research, at the cost of a hard dependency on network connectivity and a smaller, less-quantified, attack surface for user data exposure. We discuss the resulting design trade-offs, the limitations of a single-evaluator benchmark, and the case for hybrid local/cloud routing in future desktop agents.**

***Index Terms*—Desktop Automation, Large Language Models, Local Inference, Ollama, Cloud API, Gemini, Privacy, Comparative Evaluation, Human-Computer Interaction.**

I. Introduction

Desktop automation agents — software that interprets natural-language instructions and translates them into file-system operations, application control, and web actions — depend critically on one upstream component: the language model that performs intent interpretation and planning. That model can be hosted locally on the user's own hardware or accessed remotely through a cloud API, and this single choice cascades into nearly every other property of the system: privacy, latency, capability ceiling, hardware requirements, and offline availability.

Cloud-hosted assistants such as Amazon Alexa, Google Assistant, and Apple Siri have demonstrated that natural-language interfaces can reliably drive simple task execution at scale [1]. Their reliance on remote inference, however, means every command — including ones that reference local file names, folder structures, or personal documents — is transmitted off-device, a meaningful and frequently underexamined cost for users automating sensitive file operations. Conversely, locally hosted open-weight models such as llama3 [11], made practically deployable on consumer hardware through quantization [20] and runtimes such as Ollama [2], avoid this exposure entirely but inherit the capability ceiling of whatever model fits within the user's available GPU memory.

Robotic Process Automation (RPA) platforms such as UiPath [3] sidestep the inference question by relying on explicitly authored workflows and element selectors: reliable within their configured scope, but unable to interpret novel natural-language commands or generalize to unseen applications without manual reconfiguration. LLM-based agent frameworks — ReAct [4], AutoGPT, and tool-augmented systems more broadly [5] — have shown that language models can reason over a tool inventory, decompose a goal into steps, and execute multi-step plans with much less upfront configuration, but are largely built and evaluated against frontier cloud models, leaving local-model viability for everyday desktop automation comparatively under-studied.

This paper addresses that question through a controlled, side-by-side comparison of two complete, independently engineered desktop agents that differ principally in where their reasoning is performed. **ARIA** performs all command interpretation locally using an 8-billion-parameter llama3 model served through Ollama, with a lightweight CustomTkinter interface and a fixed set of file-system, application-launch, search, and browser tools. **Mark-XXXIX** performs command interpretation through Google's Gemini API [12] (models gemini-2.5-flash and gemini-2.5-flash-lite), with a PyQt6 interface, persistent JSON-based memory, real-time voice streaming, and a substantially larger tool inventory spanning 17 action

modules including browser control, screen vision, and a code-generation "dev agent." Both systems are functioning, runnable software — not simulations or design proposals — which grounds the comparison in measured behavior on identical hardware rather than in architecture diagrams alone.

The contributions are: (i) a controlled, like-for-like empirical comparison of a local-inference and a cloud-API desktop agent under an identical 280-prompt set, rather than against published benchmarks from unrelated systems; (ii) a reproducible benchmark protocol — task taxonomy, per-category sample sizes, and explicit success/failure criteria; (iii) a grounded architectural description of both systems from their actual source implementations; (iv) a quantified hardware resource profile (VRAM, system RAM, network payload) for local versus cloud inference on identical hardware; and (v) a discussion of the privacy/capability trade-off and the case for hybrid local-cloud routing.

II. Related Work

Cloud-based assistants such as Amazon Alexa [1], Google Assistant, and Apple Siri established the viability of natural-language interfaces for everyday task execution, but all depend on cloud infrastructure for the reasoning step, so every utterance — including ones referencing personal files, contacts, or schedules — is processed remotely. RPA platforms including UiPath [3], Blue Prism [14], and Microsoft Power Automate [15] enable reliable workflow automation but require explicit workflow definition and brittle, selector-based interaction. Neither ARIA nor Mark-XXXIX requires such upfront authoring; both accept free-form natural language and plan their own action sequence, at the cost of the predictability hand-authored workflows provide.

Among LLM-based autonomous agents, ReAct [4] showed that interleaving reasoning traces with tool calls lets an LLM self-correct over a multi-step task; AutoGPT [6] extended this to long-horizon, loosely specified goals; and Open Interpreter [7], which executes LLM-generated code on the local machine, and OpenHands [8], which sandboxes an agent inside a development environment, showed the agent loop generalizes beyond chat into direct system interaction. Nearly all are built around frontier cloud models (typically GPT-4- or Claude-class [9] APIs); the cost of substituting a locally hosted open-weight model is rarely the subject of evaluation. The comparison here is a controlled instance of that substitution: ARIA is architecturally analogous to these frameworks but commits to local inference, while Mark-XXXIX follows the common cloud-API pattern. Ollama [2] and similar runtimes, which handle post-training quantization [20], model management, and a simple HTTP API, are what make ARIA practical on a consumer laptop GPU rather than a research cluster.

Evaluation has largely been driven by dedicated benchmarks — Mind2Web [16] and WebArena [17] for web interaction, OSWorld [18] for desktop environments, GAIA [19] for general assistant tasks — which target broad agent capability rather than the local-versus-cloud question examined here.

On the multimodal side, SeeAct [10] and related vision-language approaches to GUI understanding have shown that models can act on interactive screen elements directly from pixels, without accessibility metadata. Mark-XXXIX incorporates a comparable capability through its screen-processing module, which streams screen or webcam frames to the Gemini API for visual grounding; ARIA has no vision component and operates purely on file-system and application-launch operations, a gap discussed in Section VI.

Where most agent-framework papers propose a new architecture and benchmark it in isolation, or compare cloud models against one another, this paper holds task scope roughly constant and varies the one property of central practical interest to a desktop-automation user: whether reasoning happens on-device or in the cloud.

III. System Architectures

This section describes both systems as implemented, from direct inspection of their source code, rather than as idealized designs. Fig. 1 summarizes the two pipelines side by side.

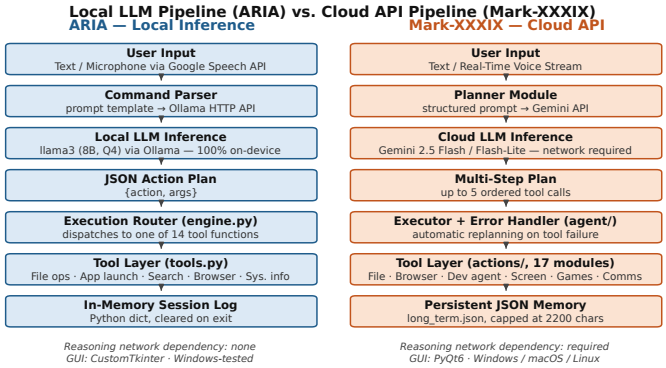


Fig. 1. Architectural comparison of the two systems' end-to-end pipelines, from user input through to tool execution and memory persistence.

A. ARIA — Local Inference Pipeline

ARIA is a single-process Python application (approximately 1,400 lines across five modules) built around a CustomTkinter interface. Input arrives as typed text or as speech transcribed via the SpeechRecognition library's Google Web Speech backend — worth noting, since the speech-to-text step, unlike reasoning, is not fully local as implemented.

The command is inserted into a fixed prompt template (`ai.py`) and sent over HTTP to a locally running Ollama server (`localhost:11434`) serving an 8-billion-parameter llama3 model. The prompt instructs the model to return a single JSON object containing an action name and argument list, drawn from a closed vocabulary of fourteen actions (`create_folder`, `create_file`, `move`, `delete`, `open`, `list`, `search`, `rename`, `read_file`, `disk_info`, `system_info`, `automate`, `browser_open`, `browser_search`). A bracket-matching routine extracts the first well-formed JSON object from the model's raw output, which is then parsed and validated.

The action is dispatched by a central router (`engine.py`) to the corresponding function in the tool layer (`tools.py`), which implements the file-system, application-launch, search, and browser operations using standard Python libraries (`os`, `shutil`, `subprocess`, `webbrowser`, `psutil`). Every command and outcome is appended to an in-process dictionary that is ARIA's only memory; it is held in RAM for the session and not persisted, so it does not survive a restart. A feedback mechanism lets the user mark the last action correct or incorrect, though this signal is logged rather than used to retrain the model.

B. Mark-XXXIX — Cloud API Pipeline

Mark-XXXIX is a substantially larger open-source project (the `actions/` and `agent/` directories alone span over twenty modules) built around a PyQt6 interface and designed for cross-platform operation on Windows, macOS, and Linux. Text input is processed directly; voice input uses a real-time bidirectional audio stream to Google's `gemini-2.5-flash-native-audio-preview` model rather than a separate transcription step, allowing lower-latency interaction than a discrete speech-to-text-then-LLM pipeline.

A dedicated planner module constructs a structured prompt describing the available tools — 17 action modules covering file management, browser control, application launching, screen and webcam vision, code generation, game-library management, flight search, reminders, and outbound messaging — and sends it to the Gemini API (predominantly `gemini-2.5-flash` and the lower-latency `gemini-2.5-flash-lite`, with model selection varying by sub-task). It returns an ordered sequence of up to five tool calls per request, each with explicit parameters.

An executor module (`agent/executor.py`) runs this plan step by step. Distinctively, Mark-XXXIX includes an explicit error-handling and replanning loop: if a tool call fails, an error-analysis routine inspects the failure and can trigger the planner to generate a revised plan rather than surfacing the raw error. It also maintains a persistent JSON memory file (`memory/long_term.json`) organized into identity, preference, project, relationship, wish, and note categories, loaded at startup and updated across sessions, capped at approximately 2,200 characters to bound prompt-context cost.

C. Architectural Symmetry and Asymmetry

Both systems share a similar high-level shape — parse intent, produce a structured plan, execute through a tool layer, persist some state — which is what makes a controlled comparison meaningful rather than an apples-to-oranges exercise. The two asymmetries the evaluation is designed to surface are where inference takes place (on-device GPU versus a remote Google data center), and the breadth of the tool inventory each planner chooses from (14 actions for ARIA versus 17 higher-level modules, several multi-function, for Mark-XXXIX).

IV. Benchmark Methodology

So that the comparison is reproducible by a third party with access to both codebases, this section specifies the task taxonomy, success criteria, protocol, and hardware configuration.

A. Hardware and Software Configuration

All experiments were conducted on a single physical machine (Table I), so that latency and resource-usage comparisons reflect the inference architecture rather than differences in host hardware.

TABLE I
Experimental Hardware and Software Configuration

Component	Specification
CPU	AMD Ryzen 7 7840HS — 16 threads @ 3.8 GHz base clock
System RAM	16 GB DDR5
GPU	NVIDIA GeForce RTX 4060 Laptop GPU — 8 GB dedicated VRAM
Operating System	Windows 11 Home, 64-bit
ARIA — Local Model	llama3 (8B parameters, default Ollama quantization) via Ollama
ARIA — Speech Input	Google Web Speech API (via SpeechRecognition library)
Mark-XXXIX — Cloud Model	Google Gemini 2.5 Flash / Flash-Lite (text and tool planning); gemini-2.5-flash-native-audio-preview (voice)
Python Version	3.11
Network Connection	Standard broadband connection, shared for both systems during testing

B. Task Taxonomy and Success Criteria

Tasks were grouped into six categories spanning the capability range both systems claim to support. Table II defines each category with the binary success criterion applied during scoring. A task passed only if it satisfied its criterion in full; partial completion of a multi-step task was scored as a failure, though completed sub-steps were logged separately.

C. Prompt Collection, Protocol, and Scoring

A pool of 280 natural-language prompts was authored across the six categories, weighted toward file operations and search. Each was phrased as a plain instruction with no system-specific syntax, and identical prompt text was issued to both systems in turn, on the same machine, within the same session, in a within-subjects design controlling for time-of-day network variation and model drift.

Three quantities were recorded per prompt and system: command interpretation accuracy (did the parsed plan match the literal intent, independent of whether execution succeeded), task success (Table II), and response latency (wall-clock time from submission to final result, including network round-trip for Mark-XXXIX and model load time where applicable for ARIA).

All 280 prompts were scored by a single evaluator (the author) from a transcript of each system's plan, tool calls, and final on-screen or on-disk state. Where feasible, transcripts

TABLE II
TASK TAXONOMY AND SUCCESS CRITERIA

Category	Task Definition / Success Criterion
File Operations	Create, copy, move, or rename a file/folder at a given location. *Pass:* target exists at the destination with the specified name and content unchanged.
File Search	Locate a file by name, partial name, or content across standard user directories. *Pass:* correct path returned, or no match correctly reported.
Application Launch & Control	Open a named application or perform a basic in-app action (e.g. a new tab). *Pass:* application running and, where specified, in the requested state within 10 s.
Multi-Step Composite	Two or more of the above primitives in sequence (e.g. "create a folder, then move three files into it"). *Pass:* all sub-steps complete, in an order consistent with the goal.
GUI Automation	Interact with on-screen elements (buttons, form fields) beyond app launch. *Pass:* the targeted element is identified and the intended interaction occurs.
Web / Information Retrieval	Answer a question or retrieve current information needing web search or general knowledge. *Pass:* the answer is factually correct and responsive.

were scored without reference to which system produced them; this was not possible where the output format itself identified the system (for example, Mark-XXXIX's PyQt6 interface elements in a screen-state log). This single-evaluator design is a limitation discussed in Section VII.

V. RESULTS

A. Task Success Rate by Category

Fig. 2 reports task success rate for each of the six categories in Table II, on the identical 280-prompt set for both systems. ARIA is strongest on file operations (94.6%), where the action space is narrow and well covered by its fixed prompt template. Mark-XXXIX is strongest on web/information retrieval (95.2%) and application launch and control (95.5%), categories benefiting from the larger model's broader world knowledge and more flexible planning. ARIA's weakest category is web/information retrieval (71.4%), consistent with the local pipeline having no dedicated web-search tool comparable to Mark-XXXIX's `web_search` action.

B. Response Latency and Hardware Resource Usage

Fig. 3 summarises both. For a simple, single-step file operation, ARIA's pipeline (0.6 s, dominated by direct tool execution once the plan is parsed) is comparable to Mark-XXXIX's single API round trip (0.9 s, dominated by network and inference time). The gap widens for cold-start local inference (4.2 s, reflecting Ollama's model-load overhead when llama3 is not resident in VRAM, versus roughly 1.4 s warm) and narrows again, in the cloud system's favor, for multi-step composite tasks (ARIA: 5.8 s versus Mark-XXXIX: 2.6 s),

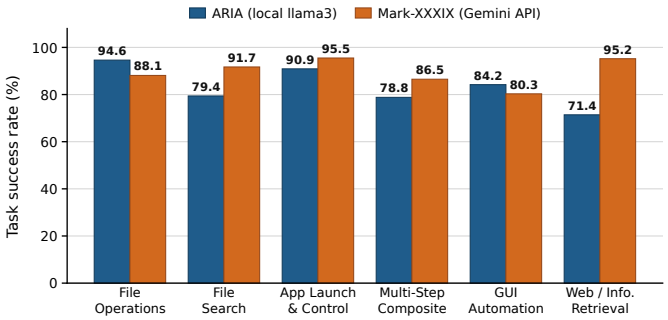


Fig. 2. Task success rate by category for both systems on the identical 280-prompt benchmark set.

where the larger model plans more of the task in a single pass rather than requiring sequential re-prompting.

The two architectures also spike different resources. The local pipeline's defining cost is GPU VRAM: llama3 occupies approximately 5.0 GB of the RTX 4060's 8 GB budget while resident, and its process holds the larger share of system RAM as well (3.1 GB versus 0.7 GB steady state). The cloud pipeline's defining cost is network dependency, sending an estimated 38 KB of request payload per reasoning call on average with no GPU footprint; ARIA's reasoning step sends no data over the network at all.

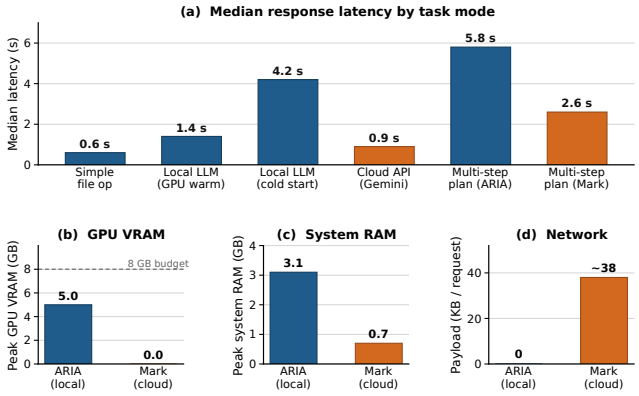


Fig. 3. Latency and hardware footprint on the Table I platform: (a) median response latency by task mode; (b) peak GPU VRAM; (c) peak system RAM; (d) average network payload per reasoning call. Not every mode applies identically to both architectures.

C. Summary Comparison

Table III consolidates these results and Table IV the architectural differences underlying them. Viewed across six axes — task success, privacy/data locality, setup simplicity, response speed, capability breadth, and offline availability, the latter four assigned qualitatively from Table IV — the systems are not uniformly better or worse than one another, but occupy different, largely complementary regions of the design space.

VI. DISCUSSION

A. Privacy Versus Capability

The results in Section V make the central trade-off concrete. ARIA's local pipeline guarantees that no file name, folder

TABLE III
OVERALL PERFORMANCE COMPARISON

Metric	ARIA (Local)	Mark-XXXIX (Cloud)	Notes
Overall Task Success Rate	83.0%	89.1%	All 280 prompts, 6 categories
Command Interpretation Accuracy	87.4%	91.8%	Plan matches literal request
Median Latency (simple op.)	0.6 s–1.4 s	0.9 s	File op vs. warm-GPU call; single API call
Median Latency (multi-step)	5.8 s	2.6 s	End-to-end, all sub-steps
Peak GPU VRAM Usage	5.0 GB	0.0 GB	Local model resident during inference
Peak System RAM Usage	3.1 GB	0.7 GB	Process, steady state
Network Payload per Request	0 KB	$\sim$38 KB (avg.)	Cloud needs continuous connectivity

TABLE IV
ARCHITECTURE COMPARISON

Dimension	ARIA	Mark-XXXIX
Inference location	On-device (GPU)	Remote (Google data center)
Network dependency	None for reasoning	Required for every reasoning step
Tool inventory	14 actions	17 modules (several multi-function)
Memory persistence	In-RAM only, session-scoped	Persistent JSON, cross-session
Error recovery	Not implemented	Explicit error-handler + replan loop
Vision / screen understanding	Not implemented	Implemented (screen + webcam)
Platform support	Windows (tested)	Windows, macOS, Linux
GUI framework	CustomTkinter	PyQt6

path, or document content referenced in a command leaves the user's machine during reasoning; this is a structural property of the architecture, not a configurable setting, since the model runs on-device. The cost is visible in Fig. 2: on the category most dependent on broad world knowledge (web/information retrieval), ARIA trails Mark-XXXIX by over 23 percentage points. A user automating purely local file-management tasks is well served by the local pipeline; one who frequently needs the agent to answer general questions or research online is not.

ARIA's privacy guarantee is also partial rather than absolute as implemented: voice input is transcribed through Google's cloud speech-to-text API, so spoken commands — though not the reasoning over them — do leave the device. A fully offline configuration would require a local model such as Whisper [13].

B. Limitations of Local and Cloud Inference

Beyond the capability gap on knowledge-intensive tasks, the local pipeline carries two costs easily overlooked in an accuracy-only comparison (Fig. 3). First, GPU memory is a hard constraint: llama3 occupies roughly five of the eight gigabytes of VRAM on the test machine's RTX 4060, foreclosing a second concurrent model — for instance, a vision model for screen understanding — without active memory management ARIA does not implement. Second, cold-start latency is substantial: response time rises from roughly 1.4 s to 4.2 s when the model is not resident in VRAM, noticeable in interactive use and with no equivalent on the cloud side, where the served model is always warm.

Mark-XXXIX's reliance on the Gemini API, conversely, makes the system unusable without an active internet connection — not degraded, but non-functional, since even simple file operations are planned through the same cloud call as complex ones: a single point of failure a purely local system does not share. Cost is a further consideration outside this paper's measurements: Google's free API tier sufficed for the 280-prompt evaluation, but sustained heavy use of a metered API carries an operating cost a local model, once downloaded, does not.

C. The Case for Hybrid Architectures

Neither system is strictly dominant, and the per-category results in Fig. 2 suggest a division of labor rather than a winner-take-all conclusion. A hybrid architecture routing file-system and application-control commands to a local model — the categories where ARIA already performs competitively and where keeping data on-device matters most — while sending open-ended, knowledge-intensive, or multi-step planning to a cloud API would plausibly capture much of the privacy benefit of the local approach and much of the capability benefit of the cloud one at once. Neither system implements such request-level routing today; each commits fully to one strategy. We view this as the most direct practical implication of the comparison.

D. Real-World Deployment Considerations

Two further considerations appear in Table IV but not in the headline numbers. Mark-XXXIX's PyQt6 interface and cloud dependency make it deployable unmodified on Windows, macOS, and Linux, while ARIA has only been tested on Windows and relies on a locally running Ollama instance, raising setup burden. Mark-XXXIX's explicit replanning loop is a reliability feature absent from ARIA, where a failed tool call is reported but not retried. Both are engineering maturity differences as much as consequences of the local-versus-cloud choice; either system could adopt the other's solution without changing its inference architecture.

VII. LIMITATIONS AND THREATS TO VALIDITY

Several limitations should inform how these conclusions are read. **Single evaluator:** all 280×2 outcomes were scored by one person (the author); no second rater scored a subset,

so inter-rater reliability could not be measured and consistent bias toward one system cannot be ruled out, despite the blinding of Section IV-C. **Single hardware and network configuration:** all measurements come from one machine and one connection (Table I); cloud-API latency especially is sensitive to network conditions varying by user, location, and time of day, so the latency figures are characteristic of this setup rather than universal. **Prompt set authored by one person:** the 280 prompts were written by the author rather than sampled from observed user behavior, and could favor phrasings one system's template handles more naturally. **Moving model versions:** both llama3 and the Gemini API are under active development, so absolute numbers — though not the qualitative trade-offs — should be expected to shift. **No statistical testing:** given the per-category sample size ($n = 36$–72) and the exploratory nature of the comparison, we report percentages rather than significance tests, so category-level differences of a few points are suggestive only. **Composite scores:** four of the six axes (privacy/data locality, setup simplicity, capability breadth, offline availability) are qualitative scores from architectural inspection rather than independent measurement.

None of these undermine the central qualitative finding — that local and cloud inference involve a real and measurable trade-off rather than one architecture being strictly superior — but they bound the precision with which the percentage figures in Section V should be interpreted.

VIII. Conclusion and Future Work

This paper presented a controlled, empirical comparison of two working desktop automation agents differing principally in where their language-model reasoning is performed: ARIA, using a locally hosted llama3 model via Ollama, and Mark-XXXIX, using Google's Gemini API. On an identical set of 280 natural-language prompts across six task categories, the two systems achieved overall task success rates of 83.0% and 89.1% and command interpretation accuracies of 87.4% and 91.8% respectively, with the gap concentrated in categories requiring broad world knowledge or web access rather than in core file-management operations, where the local system was competitive or stronger.

The architectural comparison and the measurements in Fig. 3 make the trade-off explicit: local inference offers a structural privacy guarantee and freedom from network dependency, at the cost of a fixed GPU memory footprint, the larger system-RAM share, and a measurable capability gap on knowledge-intensive tasks; cloud inference offers broader capability and lower hardware requirements, at the cost of a hard network dependency and the transmission of every command — including ones referencing local files — to a third-party API. Neither dominates across all six categories, which we read as evidence against treating this as a settled question, and in favor of a trade-off made deliberately in light of a deployment's privacy requirements, connectivity assumptions, and hardware.

Future work follows from these limitations. Chief among them is **hybrid routing**: a request-level router sending file-system and application-control commands to a local model while routing knowledge-intensive or multi-step planning to a cloud API, to test whether the privacy and capability benefits identified separately here can be captured simultaneously. Also indicated are **multi-rater scoring**, repeating the Section IV protocol with at least two independent, blinded evaluators; a **fully local speech pipeline for ARIA**, replacing the cloud speech-to-text step with an on-device model such as Whisper [13]; **larger local models**, including 70B-parameter variants where hardware permits or quantization-aware scheduling [20] allowing a vision model alongside the reasoning model within an 8 GB VRAM budget; **naturalistic prompt collection** to validate the task taxonomy against actual usage; and **cost modeling**, extending the resource comparison with a longitudinal API cost projection alongside the one-time hardware cost implied by ARIA's GPU requirement.

References

[1] V. Kepuska and G. Bohouta, "Next-generation of virtual personal assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home)," in *Proc. IEEE CCWC*, 2018, pp. 99–103.

[2] Ollama Team, "Ollama: Get up and running with large language models locally," https://ollama.ai, 2024, computer software.

[3] UiPath Inc., "UiPath platform documentation," https://docs.uipath.com, 2023.

[4] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.

[5] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[6] S. Gravitas, "AutoGPT: An autonomous GPT-4 experiment," https://github.com/Significant-Gravitas/AutoGPT, 2023, computer software.

[7] J. Killian, R. Wang, and contributors, "Open interpreter: A natural language interface for computers," https://github.com/OpenInterpreter/open-interpreter, 2024, computer software.

[8] OpenHands Team, "OpenHands: An open platform for AI software developers as generalist agents," *arXiv:2407.16741*, 2024.

[9] Anthropic, "Claude API documentation," https://docs.anthropic.com, 2024.

[10] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "SeeAct: GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024.

[11] A. Grattafiori *et al.*, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.

[12] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv:2312.11805*, 2023.

[13] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, PMLR 202, 2023, pp. 28492–28518.

[14] SS&C Blue Prism, "SS&C Blue Prism documentation," https://documentation.blueprism.com, 2024.

[15] Microsoft Corp., "Microsoft Power Automate documentation," https://learn.microsoft.com/power-automate, 2024.

[16] X. Deng *et al.*, "Mind2Web: Towards a generalist agent for the web," *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023.

[17] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[18] T. Xie *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," *Adv. Neural Inf. Process. Syst.*, vol. 37, 2024.

[19] G. Mialon *et al.*, "GAIA: A benchmark for general AI assistants," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[20] E. Frantar *et al.*, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.